

NOAI 2026 数学基础专项教程：从概率到梯度

本教程专为 NOAI 竞赛学生编写，系统讲解机器学习和深度学习背后的数学基础。你不需要成为数学家，但理解这些核心概念会让你在考试中游刃有余，在编程题中知其所以然。

目录

- [第一章 概率与统计基础](#)
- [第二章 常见概率分布](#)
- [第三章 线性代数基础：向量与矩阵](#)
- [第四章 微积分入门：导数与偏导数](#)
- [第五章 梯度下降：用数学优化模型](#)
- [第六章 激活函数的数学原理](#)
- [第七章 损失函数：衡量模型好坏的标尺](#)
- [第八章 反向传播：链式法则的工程应用](#)
- [第九章 卷积运算的数学本质](#)
- [第十章 排序算法与查找算法](#)
- [综合练习题](#)
- [附录 公式速查表](#)

第一章 概率与统计基础

1.1 为什么机器学习需要概率？

机器学习的本质是从不确定的数据中学习规律。现实世界的的数据充满噪声和随机性——同一张猫的照片在不同光线下看起来完全不同，同一个病人的检测结果可能时阳时阴。概率论为我们提供了一套严谨的语言来描述和处理这种不确定性。

当一个分类模型输出"这张图片是猫的概率为 0.92"时，它并不是在说"我确定这是猫"，而是在说"根据我学到的规律，这张图片有 92% 的可能性是猫"。理解这一点，是理解整个机器学习框架的起点。

1.2 基本概率概念

概率的定义：一个事件 A 发生的概率记为 $P(A)$ ，取值范围是 $[0, 1]$ 。 $P(A) = 0$ 表示不可能发生， $P(A) = 1$ 表示必然发生。

互补事件：事件 A 不发生的概率为 $P(A') = 1 - P(A)$ 。这个简单的公式在考试中经常用到——有时候"不发生"比"发生"更容易计算。

独立事件：如果两个事件互不影响，则它们同时发生的概率等于各自概率的乘积：

$$P(A \cap B) = P(A) \times P(B) \quad (\text{仅当 } A、B \text{ 独立时成立})$$

例题 1.1：一个垃圾邮件过滤器的准确率为 95%（即正确分类的概率为 0.95）。如果连续处理 3 封独立的邮件，3 封都分类正确的概率是多少？

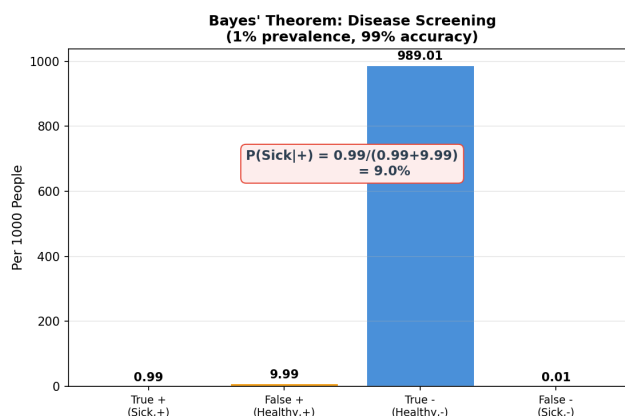
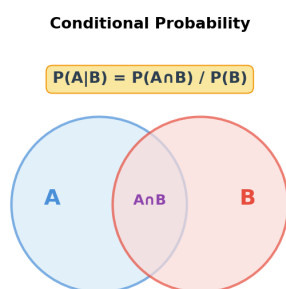
解答：由于每封邮件的分类是独立事件，所以 $P(3 \text{ 封都正确}) = 0.95 \times 0.95 \times 0.95 = 0.95^3 \approx 0.857$ 。注意，即使单次准确率很高，连续多次全部正确的概率会显著下降——这就是为什么我们需要追求更高的准确率。

1.3 条件概率

条件概率回答的是"在已知某个条件下，另一个事件发生的概率是多少"。

$$P(A|B) = P(A \cap B) / P(B)$$

读作：在 B 已经发生的条件下，A 发生的概率。



上图左侧的韦恩图直观展示了条件概率的含义： $P(A|B)$ 就是在 B 的范围内， $A \cap B$ 所占的比例。

例题 1.2：某班 40 名学生中，20 人喜欢数学，15 人喜欢编程，8 人两者都喜欢。随机选一个喜欢编程的学生，他也喜欢数学的概率是多少？

解答： $P(\text{数学}|\text{编程}) = P(\text{数学} \cap \text{编程}) / P(\text{编程}) = (8/40) / (15/40) = 8/15 \approx 0.533$ 。

1.4 贝叶斯定理

贝叶斯定理是条件概率的"反转"——它让我们能够从"原因→结果"的概率推导出"结果→原因"的概率：

$$P(A|B) = P(B|A) \times P(A) / P(B)$$

这个公式在机器学习中的应用极为广泛。朴素贝叶斯分类器就是直接基于这个公式构建的。

经典案例：疾病检测的陷阱

上图右侧展示了一个经典的贝叶斯问题。假设某种疾病的患病率为 1%（即 $P(\text{Sick}) = 0.01$ ），检测试剂的准确率为 99%（即 $P(+|\text{Sick}) = 0.99$ ， $P(-|\text{Healthy}) = 0.99$ ）。一个人检测结果为阳性，他真的患病的概率是多少？

在 1000 人中：

- 10 人患病，其中 9.9 人检测为阳性（真阳性）
- 990 人健康，其中 9.9 人检测为阳性（假阳性）

所以 $P(\text{Sick}|+) = 9.9 / (9.9 + 9.9) = 50\%$ （而非直觉中的 99%）。

如果准确率降为 99% 对 99%（如图中所示的更精确计算）， $P(\text{Sick}|+) \approx 9.0\%$ 。这个反直觉的结果告诉我们：当基础概率（先验概率）很低时，即使检测很准确，阳性结果的可信度也可能不高。

1.5 均值、方差与标准差

这三个统计量是描述数据分布最基本的工具。

均值（Mean） 描述数据的"中心位置"：

$$\mu = (1/n) \times \sum x_i$$

方差（Variance） 描述数据的"离散程度"——数据点离均值有多远：

$$\sigma^2 = (1/n) \times \sum (x_i - \mu)^2$$

标准差（Standard Deviation） 是方差的平方根，与原始数据具有相同的单位：

$$\sigma = \sqrt{(\sigma^2)}$$

例题 1.3：两个模型在 5 次测试中的准确率如下：

测试	模型 A	模型 B
1	0.85	0.70
2	0.87	0.95
3	0.84	0.60
4	0.86	0.90
5	0.88	0.85

解答：

模型 A: $\mu = 0.86, \sigma^2 = 0.0002, \sigma \approx 0.014$

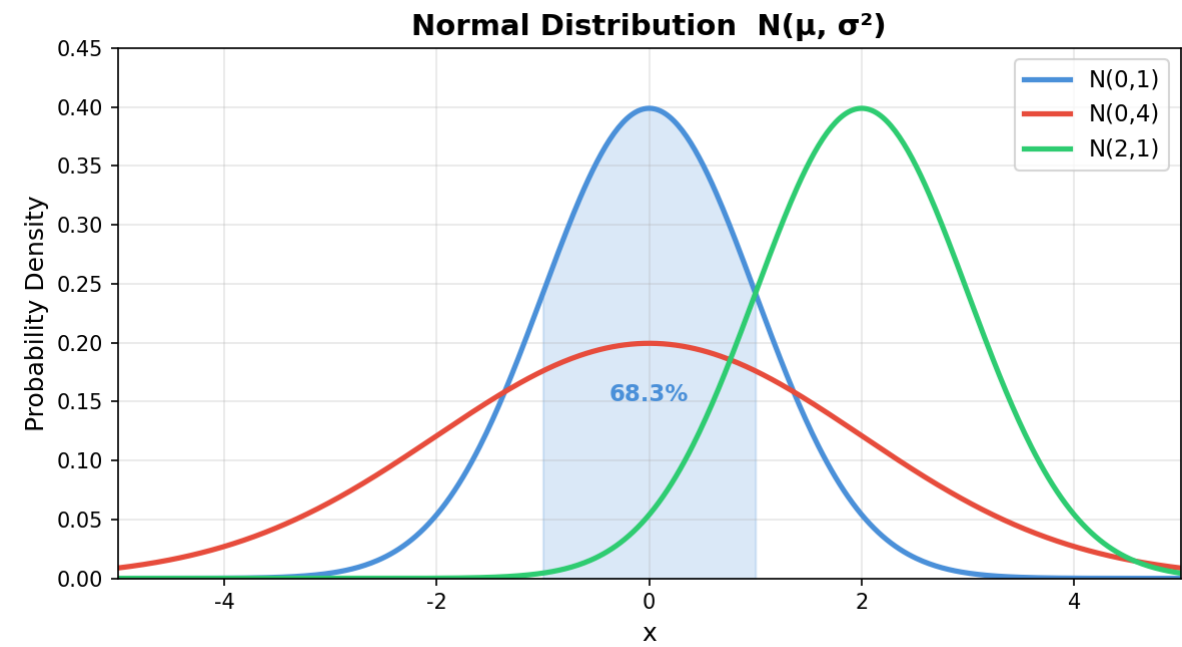
模型 B: $\mu = 0.80, \sigma^2 = 0.0164, \sigma \approx 0.128$

模型 A 的均值更高且标准差更小，说明它不仅平均表现更好，而且**更稳定**。在实际应用中，稳定性往往和准确率同样重要——你不会希望一个自动驾驶系统时好时坏。

1.6 正态分布

正态分布（也叫高斯分布）是自然界中最常见的概率分布，其概率密度函数为：

$$f(x) = (1 / (\sigma\sqrt{2\pi})) \times \exp(-(x-\mu)^2 / (2\sigma^2))$$



上图展示了三条不同参数的正态分布曲线。蓝色阴影区域表示标准正态分布 $N(0,1)$ 中 $\mu \pm \sigma$ 范围内的面积，约占总面积的 **68.3%**。

68-95-99.7 法则（三西格玛法则）：

范围	概率	含义
$\mu \pm 1\sigma$	68.3%	约 2/3 的数据落在此范围

范围	概率	含义
$\mu \pm 2\sigma$	95.4%	绝大多数数据落在此范围
$\mu \pm 3\sigma$	99.7%	几乎所有数据落在此范围

在机器学习中的应用：数据标准化（Standardization）就是将数据转换为均值为 0、标准差为 1 的标准正态分布。sklearn 中的 `StandardScaler` 做的正是这件事： $z = (x - \mu) / \sigma$ 。

例题 1.4：某数据集的特征 x 的均值为 100，标准差为 15。将 $x = 130$ 标准化后的值是多少？

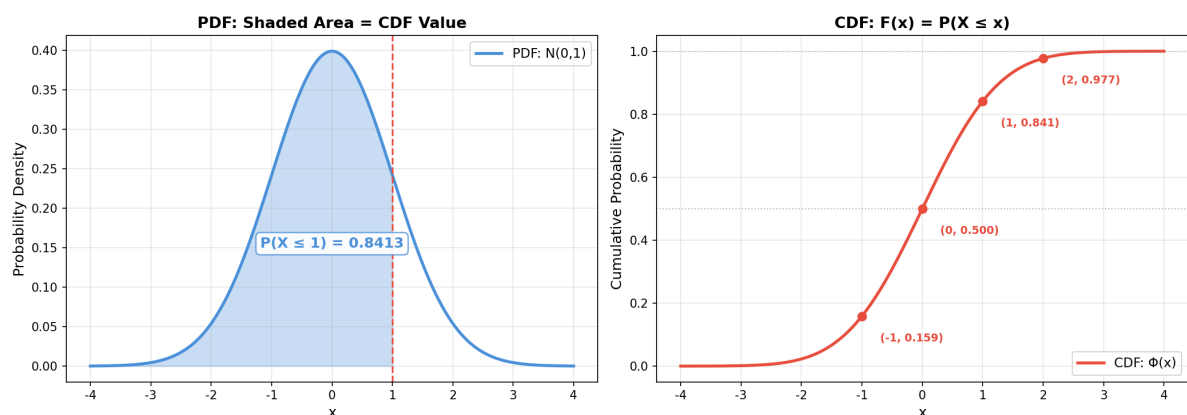
解答： $z = (130 - 100) / 15 = 30/15 = 2.0$ 。这意味着 130 在原始数据中比均值高出 2 个标准差，属于比较极端的值（只有约 2.3% 的数据会超过这个值）。

1.7 累积分布函数（CDF）

概率密度函数（PDF）告诉我们某个具体值的“密度”，而**累积分布函数（CDF）**告诉我们“小于等于某个值的概率是多少”：

$$F(x) = P(X \leq x)$$

CDF 就是 PDF 从负无穷大到 x 的积分（面积）。对于标准正态分布，这个 CDF 通常记为 $\Phi(x)$ 。



上图左侧展示了 PDF 中的阴影面积如何对应 CDF 的值：蓝色阴影区域的面积就是 $P(X \leq 1) = 0.8413$ 。右侧展示了完整的 CDF 曲线，它是一条从 0 单调递增到 1 的 S 形曲线。

CDF 的关键性质：

性质	表达式	含义
$F(-\infty) = 0$	左端趋近 0	不可能比所有值都小
$F(+\infty) = 1$	右端趋近 1	必然小于某个足够大的值
单调递增	$F(a) \leq F(b)$ 当 $a \leq b$	值越大，累积概率越高
$P(a < X \leq b)$	$F(b) - F(a)$	两点之间的概率 = CDF 之差

标准正态分布的常用 CDF 值：

x	$\Phi(x) = P(X \leq x)$	常用场景
-2	0.0228	只有 2.3% 的数据小于这个值
-1	0.1587	约 16% 的数据小于这个值
0	0.5000	恰好一半（均值处）
1	0.8413	约 84% 的数据小于这个值
2	0.9772	约 97.7% 的数据小于这个值

例题 1.5：某次考试成绩服从正态分布 $N(70, 10^2)$ 。小明考了 85 分，他超过了百分之几的同学？

解答：先标准化： $z = (85 - 70) / 10 = 1.5$ 。查 CDF 表： $\Phi(1.5) \approx 0.9332$ 。这意味着小明超过了约 **93.3%** 的同学。换句话说，只有约 6.7% 的同学比他考得更好。

例题 1.6：在标准正态分布中， $P(-1 < X \leq 2)$ 是多少？

解答： $P(-1 < X \leq 2) = \Phi(2) - \Phi(-1) = 0.9772 - 0.1587 = \mathbf{0.8185}$ 。约 81.9% 的数据落在 (-1, 2] 区间内。

第二章 常见概率分布

除了正态分布，NOAI 考试中还可能涉及另外两种重要的概率分布：二项分布和均匀分布。理解它们的特点和应用场景，能帮助你在概率相关的题目中快速识别模型。

2.1 二项分布

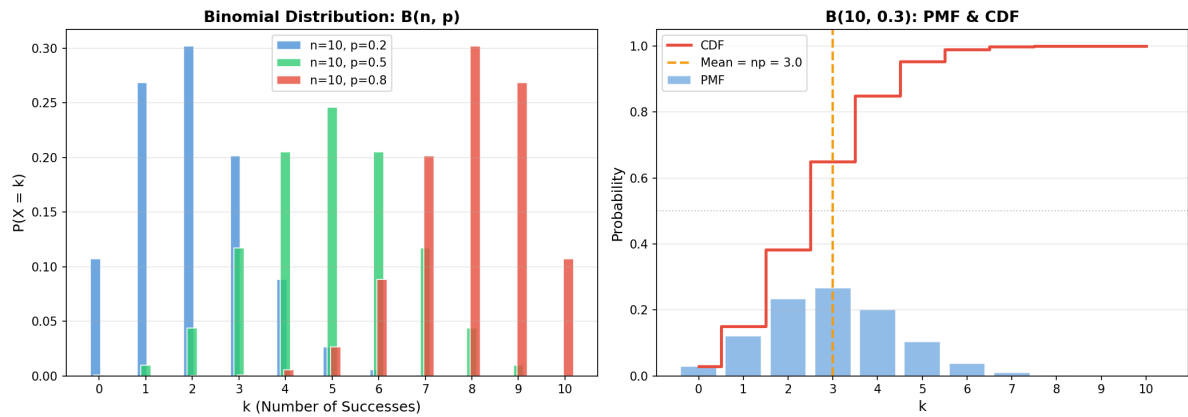
二项分布描述的是：在 n 次独立的“成功/失败”实验中，成功恰好发生 k 次的概率。每次实验的成功概率为 p 。

$$P(X = k) = C(n, k) \times p^k \times (1-p)^{n-k}$$

其中 $C(n, k) = n! / (k!(n-k)!)$ 是组合数

关键参数：

参数	公式	含义
均值	$\mu = np$	平均成功次数
方差	$\sigma^2 = np(1-p)$	成功次数的波动
标准差	$\sigma = \sqrt{np(1-p)}$	波动的度量



上图左侧展示了三种不同成功概率的二项分布（ $n=10$ ）。可以观察到：当 $p=0.5$ 时分布最对称， p 偏离 0.5 时分布变得偏斜。右侧展示了 $B(10, 0.3)$ 的 PMF（柱状图）和 CDF（阶梯线），黄色虚线标出了均值 $np = 3$ 。

例题 2.1： 一个分类器的准确率为 0.8。对 5 个独立样本进行预测，恰好有 4 个预测正确的概率是多少？

解答： 这是一个二项分布问题， $n=5, p=0.8, k=4$ 。

$$C(5, 4) = 5! / (4! \times 1!) = 5$$

$$P(X=4) = 5 \times 0.8^4 \times 0.2^1 = 5 \times 0.4096 \times 0.2 = \mathbf{0.4096}$$

例题 2.2： 一枚硬币抛 10 次，正面朝上 7 次或更多次的概率是多少？

解答： $P(X \geq 7) = P(X=7) + P(X=8) + P(X=9) + P(X=10)$

k	$C(10, k)$	0.5^{10}	$P(X=k)$
7	120	$1/1024$	$120/1024 = 0.1172$
8	45	$1/1024$	$45/1024 = 0.0439$
9	10	$1/1024$	$10/1024 = 0.0098$
10	1	$1/1024$	$1/1024 = 0.0010$

$$P(X \geq 7) = 0.1172 + 0.0439 + 0.0098 + 0.0010 = \mathbf{0.1719}$$

约 17.2% 的概率，说明连续 10 次抛硬币出现 7 次以上正面并不罕见。

2.2 均匀分布

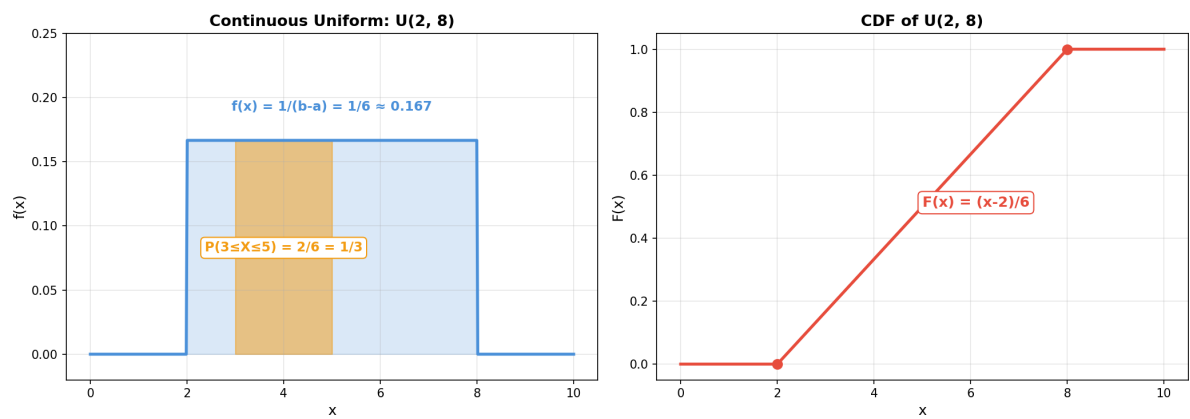
均匀分布是最简单的概率分布——在某个范围内，每个值出现的可能性完全相同。

连续均匀分布 $U(a, b)$:

$$PDF: f(x) = 1/(b-a), \text{ 当 } a \leq x \leq b; \text{ 其他位置 } f(x) = 0$$

$$CDF: F(x) = (x-a)/(b-a), \text{ 当 } a \leq x \leq b$$

参数	公式	含义
均值	$\mu = (a+b)/2$	区间中点
方差	$\sigma^2 = (b-a)^2/12$	区间越宽，方差越大



上图展示了 $U(2, 8)$ 的 PDF 和 CDF。PDF 是一条水平线（高度 $= 1/6 \approx 0.167$ ），CDF 是一条从 0 到 1 的直线。橙色阴影区域表示 $P(3 \leq X \leq 5) = (5-3)/(8-2) = 2/6 = 1/3$ 。

例题 2.3：随机数生成器产生 $[0, 1]$ 之间的均匀分布随机数。 $P(0.3 < X \leq 0.7)$ 是多少？

解答： $P(0.3 < X \leq 0.7) = F(0.7) - F(0.3) = (0.7-0)/(1-0) - (0.3-0)/(1-0) = 0.7 - 0.3 = \mathbf{0.4}$

离散均匀分布：掷一个公平的六面骰子就是离散均匀分布的经典例子。每个面的概率都是 $1/6$ ，均值 $= (1+2+3+4+5+6)/6 = 3.5$ 。

在机器学习中的应用：均匀分布常用于参数初始化。例如 PyTorch 的 `nn.init.uniform_(tensor, a, b)` 用均匀分布初始化权重。随机搜索超参数时，也经常从均匀分布中采样。

2.3 三种分布对比

分布	类型	参数	均值	方差	典型应用
正态分布 $N(\mu, \sigma^2)$	连续	μ, σ	μ	σ^2	数据标准化、误差建模
二项分布 $B(n, p)$	离散	n, p	np	$np(1-p)$	分类准确率、成功次数
均匀分布 $U(a, b)$	连续	a, b	$(a+b)/2$	$(b-a)^2/12$	参数初始化、随机采样

第三章 线性代数基础：向量与矩阵

2.1 为什么需要线性代数？

机器学习处理的数据几乎都是以矩阵的形式存在的。一张 28×28 的灰度图片就是一个 28×28 的矩阵；一个包含 1000 个样本、每个样本有 10 个特征的数据集就是一个 1000×10 的矩阵。神经网络的每一层本质上都是矩阵乘法加上激活函数。

2.2 向量与点积

向量是一组有序的数字。在机器学习中，一个样本的特征通常表示为一个向量。

点积（内积）是两个等长向量对应元素相乘再求和：

$$a \cdot b = \sum (a_i \times b_i) = a_1b_1 + a_2b_2 + \dots + a_nb_n$$

例题 2.1：计算向量 $a = [1, 2, 3]$ 和 $b = [4, 5, 6]$ 的点积。

解答： $a \cdot b = 1 \times 4 + 2 \times 5 + 3 \times 6 = 4 + 10 + 18 = 32$

点积在神经网络中无处不在——每个神经元的计算本质上就是输入向量与权重向量的点积加上偏置： $\text{output} = \mathbf{w} \cdot \mathbf{x} + \mathbf{b}$ 。

2.3 矩阵乘法

矩阵乘法是线性代数中最重要的运算。设 A 是 $m \times n$ 矩阵， B 是 $n \times p$ 矩阵，则 $C = A \times B$ 是 $m \times p$ 矩阵，其中：

$$C_{ij} = \sum (A_{ik} \times B_{kj}), \quad k \text{ 从 } 1 \text{ 到 } n$$

关键规则： A 的列数必须等于 B 的行数，结果矩阵的大小为 A 的行数 $\times$ B 的列数。

例题 2.2： 计算以下矩阵乘法。

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

解答：

$$\begin{aligned} C_{[0,0]} &= 1 \times 5 + 2 \times 7 = 5 + 14 = 19 \\ C_{[0,1]} &= 1 \times 6 + 2 \times 8 = 6 + 16 = 22 \\ C_{[1,0]} &= 3 \times 5 + 4 \times 7 = 15 + 28 = 43 \\ C_{[1,1]} &= 3 \times 6 + 4 \times 8 = 18 + 32 = 50 \\ C &= \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \end{aligned}$$

NumPy 验证：

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = A @ B # 或 np.dot(A, B) 或 np.matmul(A, B)

print(C) # [[19 22] [43 50]]
```

2.4 矩阵维度速查

在 NOAI 考试中，矩阵维度的推导是高频考点。以下是核心规则：

运算	输入维度	输出维度	条件
$A \times B$	$(m \times n) \times (n \times p)$	$m \times p$	A 的列 = B 的行
$A + B$	$(m \times n) + (m \times n)$	$m \times n$	维度必须相同
A^T (转置)	$m \times n$	$n \times m$	行列互换
逐元素乘 $A * B$	$(m \times n) * (m \times n)$	$m \times n$	维度必须相同

例题 2.3 (2024 真题模式)：一个全连接层的权重矩阵 W 的维度为 $(128, 64)$ ，输入向量 x 的维度为 $(64, 1)$ 。输出 $y = Wx$ 的维度是多少？

解答： $(128 \times 64) \times (64 \times 1) = (128, 1)$ 。这就是为什么全连接层 `nn.Linear(64, 128)` 能将 64 维输入映射到 128 维输出。

2.5 转置与对称矩阵

转置就是将矩阵的行和列互换，记为 A^T 。

$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$	$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$
--	---

转置的重要性质： $(AB)^T = B^T \times A^T$ (注意顺序反转)。

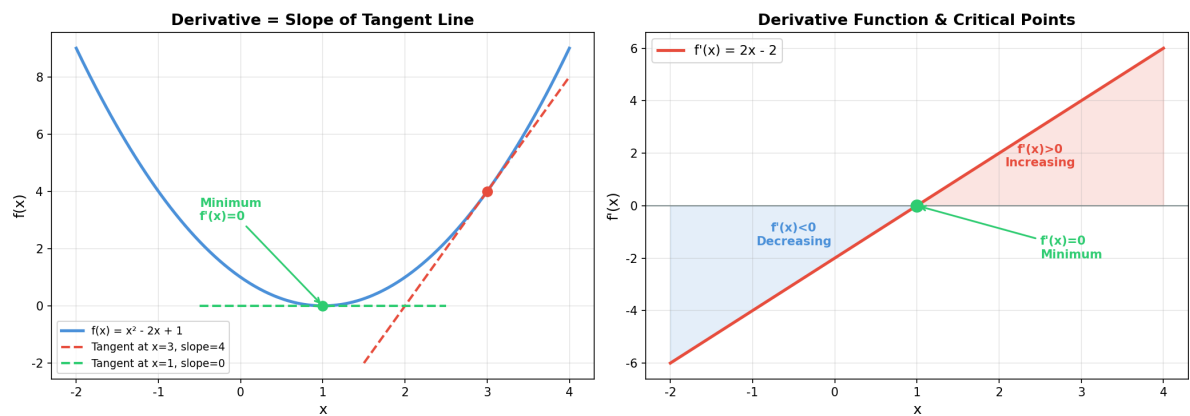
在机器学习中，转置常用于计算梯度。例如，线性回归的梯度公式中就包含 X^T ：

$$\nabla L = (1/n) \times X^T \times (Xw - y)$$

第四章 微积分入门：导数与偏导数

3.1 导数的直观理解

导数描述的是函数在某一点的变化速率。想象你在爬山——导数就是你脚下地面的坡度。坡度为正说明在上坡，坡度为负说明在下坡，坡度为零说明你站在山顶或山谷。



上图左侧展示了函数 $f(x) = x^2 - 2x + 1$ 及其在两个点的切线。在 $x=3$ 处，切线斜率为 4（陡峭上坡）；在 $x=1$ 处，切线斜率为 0（这正是函数的最小值点）。右侧展示了导数函数 $f'(x) = 2x - 2$ ，清楚地标出了导数为零的临界点。

导数的数学定义：

$$f'(x) = \lim_{\Delta x \rightarrow 0} [f(x + \Delta x) - f(x)] / \Delta x$$

这个极限的直观含义是：当我们把 x 增加一个极小的量 Δx 时， $f(x)$ 会增加多少？增加量与 Δx 的比值就是导数。

3.2 常用求导公式

以下是 NOAI 考试中最常用的求导公式，你不需要记住推导过程，但需要能熟练运用：

函数 $f(x)$	导数 $f'(x)$	记忆技巧
c （常数）	0	常数不变化
x^n	$n \times x^{(n-1)}$	指数下来乘，指数减一

函数 $f(x)$	导数 $f'(x)$	记忆技巧
e^x	e^x	自然指数的导数是自己
$\ln(x)$	$1/x$	对数的导数是倒数
$\sin(x)$	$\cos(x)$	—
$\cos(x)$	$-\sin(x)$	注意负号

例题 3.1： 求 $f(x) = 3x^2 + 2x - 5$ 的导数。

解答： 逐项求导—— $f'(x) = 3 \times 2x + 2 \times 1 - 0 = 6x + 2$

3.3 链式法则

链式法则是深度学习中最重要求导法则，没有之一。它告诉我们如何对**复合函数**求导：

如果 $y = f(g(x))$ ，则 $dy/dx = f'(g(x)) \times g'(x)$

用更直观的方式表达：

$dy/dx = (dy/du) \times (du/dx)$ ，其中 $u = g(x)$

为什么链式法则如此重要？ 因为神经网络就是一系列函数的复合。输入 x 先经过第一层变换得到 h_1 ，再经过第二层变换得到 h_2 ，最终得到输出 y 。要计算损失函数对第一层参数的梯度，就必须用链式法则一层一层地“传递”回去。

例题 3.2： 求 $f(x) = (3x + 1)^4$ 的导数。

解答： 设 $u = 3x + 1$ ，则 $f = u^4$ 。

$f'(x) = 4u^3 \times (du/dx) = 4(3x+1)^3 \times 3 = 12(3x+1)^3$

例题 3.3： 求 $\sigma(x) = 1/(1 + e^{(-x)})$ 的导数（这就是 Sigmoid 函数）。

解答：设 $u = 1 + e^{(-x)}$ ，则 $\sigma = 1/u = u^{-1}$ 。

$$d\sigma/dx = -u^{-2} \times (du/dx) = -1/(1+e^{(-x)})^2 \times (-e^{(-x)})$$

$$= e^{(-x)} / (1+e^{(-x)})^2$$

经过化简（这是一个经典的化简技巧）：

$$= [1/(1+e^{(-x)})] \times [1 - 1/(1+e^{(-x)})]$$

$$= \sigma(x) \times (1 - \sigma(x))$$

这个优美的结果意味着 Sigmoid 的导数可以用它自身来表示，这在编程实现中非常方便。

3.4 偏导数

当函数有多个变量时，我们对其中一个变量求导，同时把其他变量当作常数，这就是偏导数。

对于 $f(x, y) = x^2 + 3xy + y^2$ ：

- $\partial f / \partial x = 2x + 3y$ （把 y 当常数）
- $\partial f / \partial y = 3x + 2y$ （把 x 当常数）

例题 3.4：损失函数 $L(w, b) = (wx + b - y)^2$ ，其中 $x=2, y=5$ 。求 $\partial L / \partial w$ 和 $\partial L / \partial b$ 。

解答：设 $e = wx + b - y = 2w + b - 5$ （误差项）。

$$\partial L / \partial w = 2e \times (\partial e / \partial w) = 2(2w + b - 5) \times 2 = 4(2w + b - 5)$$

$$\partial L / \partial b = 2e \times (\partial e / \partial b) = 2(2w + b - 5) \times 1 = 2(2w + b - 5)$$

如果当前 $w=1, b=1$ ，则 $e = 2+1-5 = -2$ ，所以 $\partial L / \partial w = 4 \times (-2) = -8$ ， $\partial L / \partial b = 2 \times (-2) = -4$ 。

3.5 梯度

梯度就是所有偏导数组成的向量，记为 ∇f （nabla f ）。

$\nabla f(x, y) = [\partial f / \partial x, \partial f / \partial y]$

梯度的方向指向函数值增长最快的方向。因此，沿梯度的反方向走，就能最快地减小函数值——这就是梯度下降的核心思想。

第五章 梯度下降：用数学优化模型

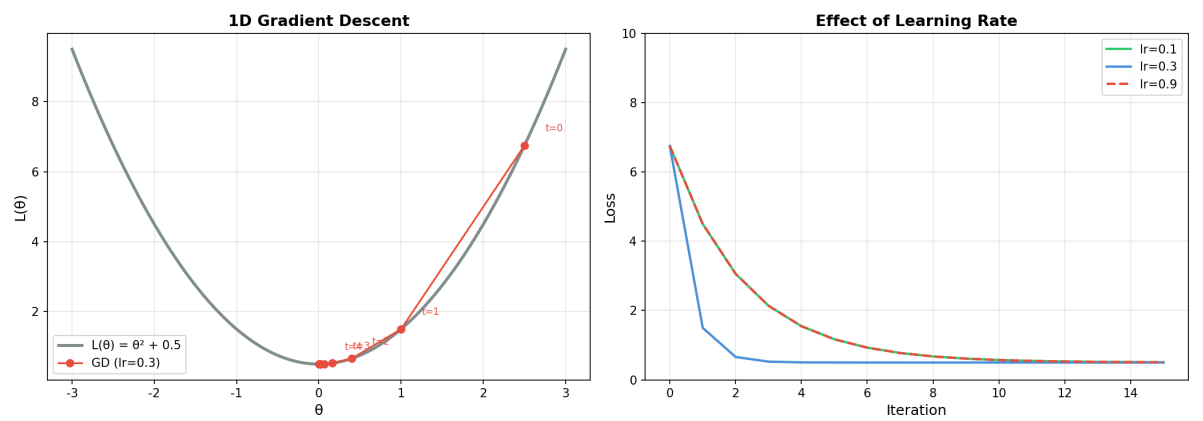
4.1 核心思想

训练机器学习模型的过程，本质上是一个优化问题：找到一组参数 θ ，使得损失函数 $L(\theta)$ 最小。梯度下降是解决这个问题最常用的方法。

参数更新公式：

$$\theta_{new} = \theta_{old} - \alpha \times \nabla L(\theta_{old})$$

其中 α 是学习率（Learning Rate），控制每一步走多远。



上图左侧展示了一维梯度下降的过程：从初始点 $\theta=2.5$ 出发，沿着损失函数的梯度反方向逐步移动，最终收敛到最小值点。右侧对比了三种学习率的效果：

学习率	表现	问题
$\alpha = 0.1$ （绿色）	缓慢但稳定地收敛	收敛太慢，需要更多迭代

学习率	表现	问题
$\alpha = 0.3$ (蓝色)	适中的速度收敛	较好的平衡
$\alpha = 0.9$ (红色虚线)	剧烈震荡	可能无法收敛

4.2 手算梯度下降

例题 4.1： 对于损失函数 $L(\theta) = \theta^2 + 2$ ，初始参数 $\theta_0 = 4$ ，学习率 $\alpha = 0.3$ 。手算前 3 步梯度下降。

解答：

$$L'(\theta) = 2\theta \quad (\text{导数})$$

步骤	θ	$L(\theta)$	$L'(\theta)$	$\theta_{\text{new}} = \theta - 0.3 \times L'(\theta)$
0	4.0	18.0	8.0	$4.0 - 0.3 \times 8.0 = \mathbf{1.6}$
1	1.6	4.56	3.2	$1.6 - 0.3 \times 3.2 = \mathbf{0.64}$
2	0.64	2.41	1.28	$0.64 - 0.3 \times 1.28 = \mathbf{0.256}$

可以看到， θ 正在逐步趋近最小值点 $\theta=0$ ，损失也在逐步减小。

4.3 多参数梯度下降

在实际的机器学习中，模型通常有多个参数。以线性回归 $y = wx + b$ 为例：

损失函数 (MSE) : $L(w, b) = (1/n) \times \sum (wx_i + b - y_i)^2$

梯度：

- $\partial L / \partial w = (2/n) \times \sum (wx_i + b - y_i) \times x_i$
- $\partial L / \partial b = (2/n) \times \sum (wx_i + b - y_i)$

更新规则：

- $w_{\text{new}} = w - \alpha \times \partial L / \partial w$

- $b_{\text{new}} = b - \alpha \times \partial L / \partial b$

例题 4.2： 给定数据点 (1, 2) 和 (2, 4)，初始参数 $w=0, b=0$ ，学习率 $\alpha=0.1$ 。执行一步梯度下降。

解答：

预测值： $\hat{y}_1 = 0 \times 1 + 0 = 0, \hat{y}_2 = 0 \times 2 + 0 = 0$

误差： $e_1 = 0 - 2 = -2, e_2 = 0 - 4 = -4$

$\partial L / \partial w = (2/2) \times [(-2) \times 1 + (-4) \times 2] = 1 \times (-2 - 8) = -10$

$\partial L / \partial b = (2/2) \times [(-2) + (-4)] = 1 \times (-6) = -6$

更新： $w = 0 - 0.1 \times (-10) = \mathbf{1.0}, b = 0 - 0.1 \times (-6) = \mathbf{0.6}$

4.4 SGD 与 Mini-batch

在实际训练中，我们通常不会用全部数据计算梯度（计算量太大），而是采用以下变体：

方法	每次使用的样本数	特点
批量梯度下降（BGD）	全部样本	梯度准确，但计算慢
随机梯度下降（SGD）	1 个样本	计算快，但梯度噪声大
小批量梯度下降（Mini-batch）	一小批样本（如 32、64）	平衡速度和稳定性

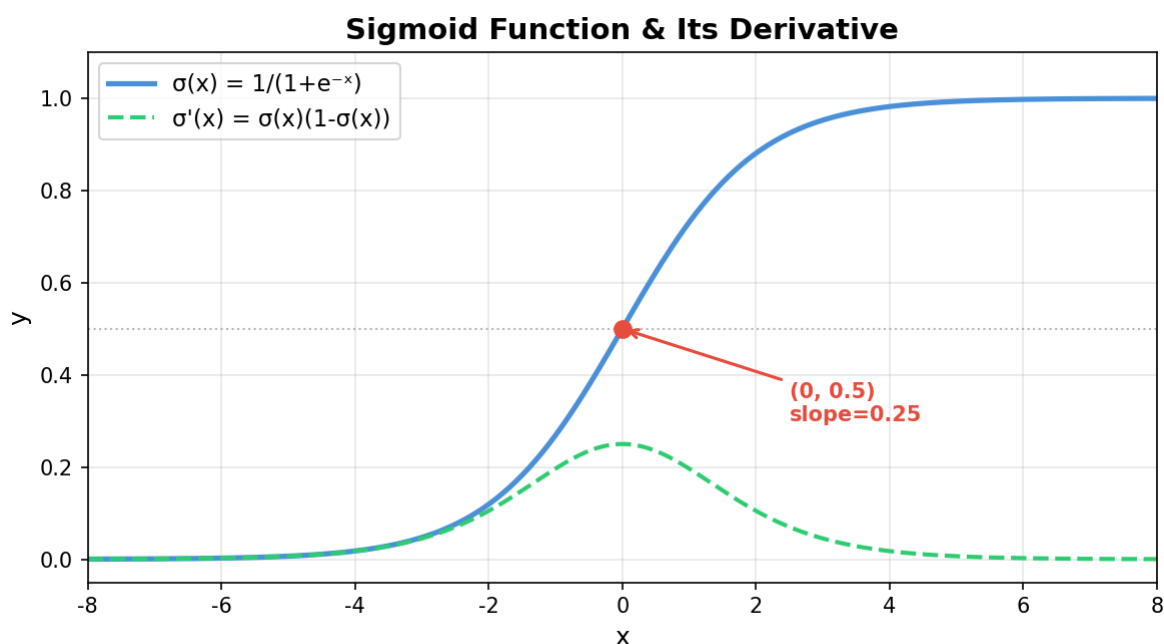
在 PyTorch 中， `DataLoader(batch_size=32)` 就是在实现 Mini-batch 梯度下降。

第六章 激活函数的数学原理

5.1 为什么需要激活函数？

如果神经网络只有线性变换（矩阵乘法 + 加法），那么无论堆叠多少层，整个网络仍然等价于一个线性变换。激活函数引入了**非线性**，使得神经网络能够拟合任意复杂的函数。

5.2 Sigmoid 函数



公式： $\sigma(x) = 1 / (1 + e^{(-x)})$

性质：

- 输出范围：(0, 1)，可以解释为概率
- $\sigma(0) = 0.5$
- 导数： $\sigma'(x) = \sigma(x) \times (1 - \sigma(x))$ ，最大值为 0.25（在 $x=0$ 处）

优点：输出可以直接作为概率使用，适合二分类的输出层。

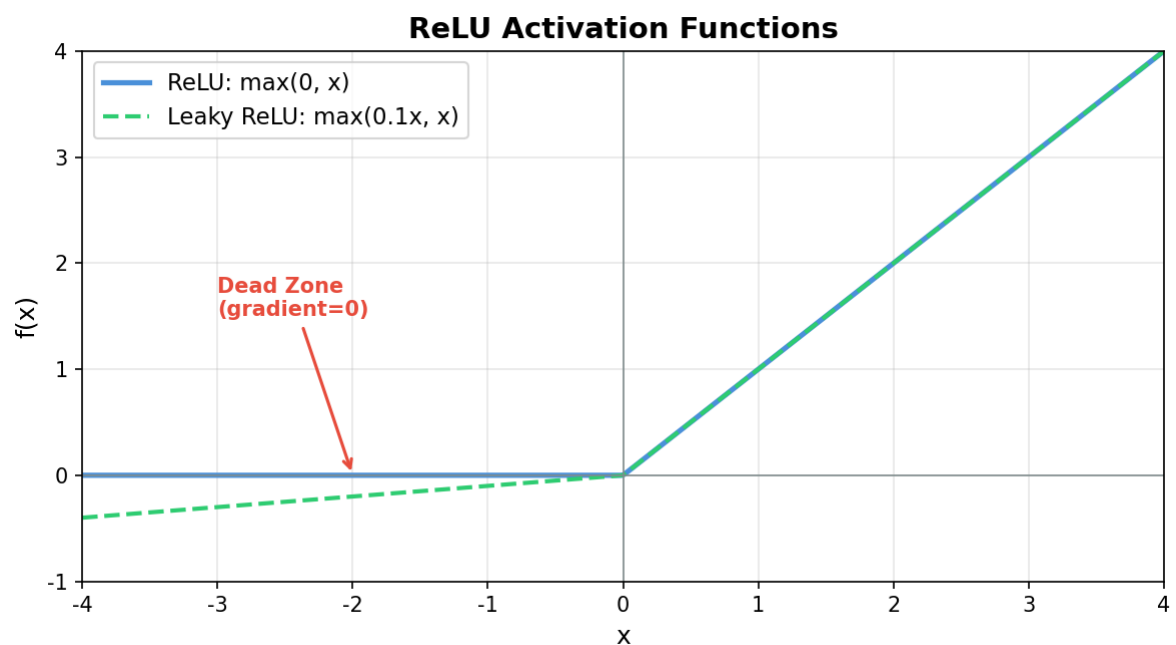
缺点：当 $|x|$ 很大时，导数接近 0（图中绿色虚线在两端趋近于零），导致**梯度消失**问题——深层网络的参数几乎无法更新。

例题 5.1：计算 $\sigma(0)$ 、 $\sigma(2)$ 、 $\sigma(-2)$ 的值和导数。

x	$e^{(-x)}$	$\sigma(x) = 1/(1+e^{(-x)})$	$\sigma'(x) = \sigma(x)(1-\sigma(x))$
0	1.000	0.500	0.250
2	0.135	0.881	0.105
-2	7.389	0.119	0.105

注意 $\sigma(2)$ 和 $\sigma(-2)$ 的导数相同，因为 Sigmoid 函数关于点 $(0, 0.5)$ 中心对称。

5.3 ReLU 函数



公式： $\text{ReLU}(x) = \max(0, x)$

性质：

- $x > 0$ 时，输出等于输入，导数为 1
- $x \leq 0$ 时，输出为 0，导数为 0 ("死亡区域")

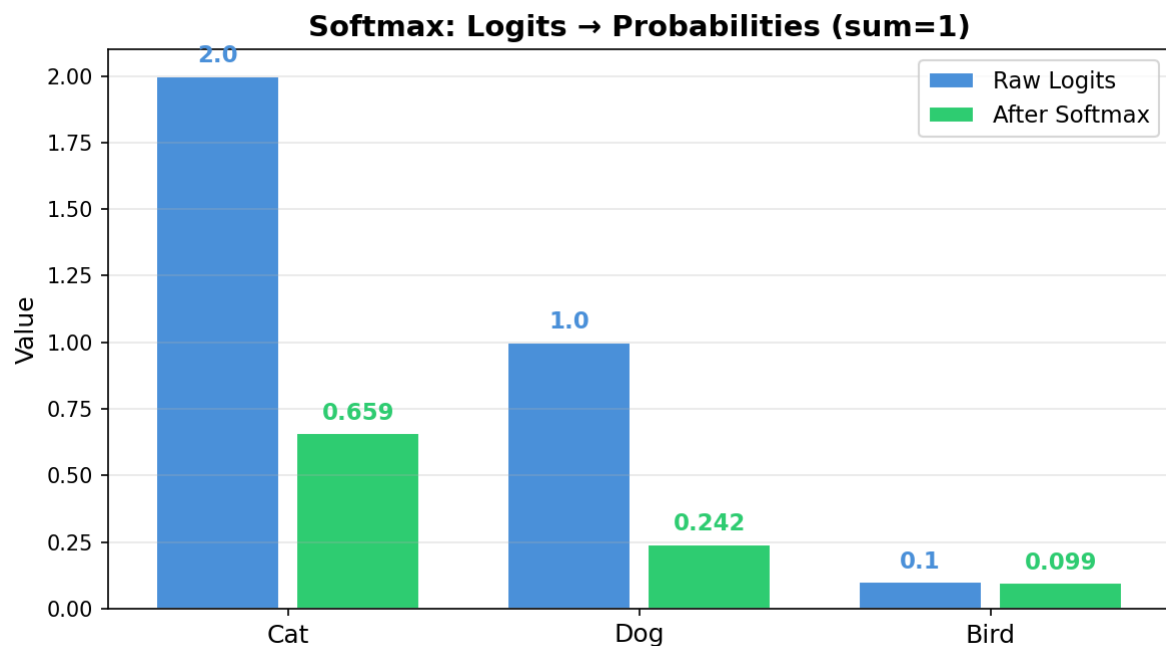
ReLU 是现代深度学习中最常用的激活函数，因为它计算简单且能有效缓解梯度消失问题。但它有一个缺陷：如果某个神经元的输入始终为负，它就永远输出 0，参数永远不会更新——这就是"死亡 ReLU"问题。

Leaky ReLU 通过在负半轴给一个小斜率（如 0.1）来解决这个问题： $\text{LeakyReLU}(x) = \max(0.1x, x)$ 。

例题 5.2： 计算 $\text{ReLU}(-3)$ 、 $\text{ReLU}(0)$ 、 $\text{ReLU}(2.5)$ 。

解答： $\text{ReLU}(-3) = \max(0, -3) = 0$ ； $\text{ReLU}(0) = \max(0, 0) = 0$ ； $\text{ReLU}(2.5) = \max(0, 2.5) = 2.5$ 。

5.4 Softmax 函数



公式： $\text{Softmax}(z_i) = e^{(z_i)} / \sum(e^{(z_j)})$

Softmax 将一组任意实数（称为 logits）转换为概率分布——每个输出都在 (0, 1) 之间，且所有输出之和为 1。

例题 5.3： 对 logits = [2.0, 1.0, 0.1] 计算 Softmax。

解答：

$e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.1} = 1.105$

总和 = $7.389 + 2.718 + 1.105 = 11.212$

Softmax = $[7.389/11.212, 2.718/11.212, 1.105/11.212] = [0.659, 0.242, 0.099]$

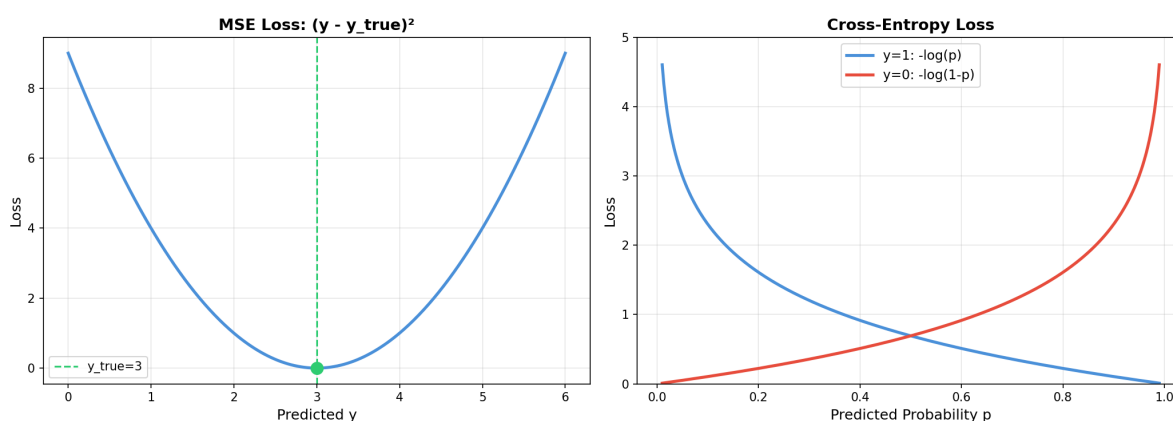
验证： $0.659 + 0.242 + 0.099 = 1.000$ （和为 1）。

如上图所示，原始 logits 中 "Cat" 的值最大（2.0），经过 Softmax 后它的概率也最高（0.659），模型预测为 "Cat"。

第七章 损失函数：衡量模型好坏的标尺

6.1 损失函数的角色

损失函数（Loss Function）量化了模型预测值与真实值之间的差距。训练的目标就是最小化损失函数。不同的任务需要不同的损失函数。



6.2 均方误差（MSE）

用于回归任务（预测连续值）。

$$MSE = (1/n) \times \sum (\hat{y}_i - y_i)^2$$

上图左侧展示了 MSE 损失的形状——它是一个以真实值为最低点的抛物线。这意味着预测值离真实值越远，损失增长越快（二次增长），对大误差的惩罚更重。

MSE 的梯度： $\partial MSE / \partial \hat{y} = (2/n) \times \sum (\hat{y}_i - y_i)$

例题 6.1： 真实值 $y = [3, 5, 7]$ ，预测值 $\hat{y} = [2.5, 5.5, 6]$ 。计算 MSE。

解答： $MSE = [(2.5-3)^2 + (5.5-5)^2 + (6-7)^2] / 3 = [0.25 + 0.25 + 1.0] / 3 = 0.5$

6.3 交叉熵损失 (Cross-Entropy Loss)

用于分类任务（预测类别概率）。

二分类交叉熵：

$$L = -[y \times \log(p) + (1-y) \times \log(1-p)]$$

其中 y 是真实标签（0 或 1）， p 是模型预测的正类概率。

上图右侧展示了交叉熵损失的两条曲线。当真实标签 $y=1$ 时（蓝色），模型预测的概率 p 越接近 1，损失越小； p 越接近 0，损失趋向无穷大。这种"对错误预测的极端惩罚"使得模型会努力给出自信且正确的预测。

多分类交叉熵：

$$L = -\sum(y_i \times \log(p_i))$$

其中 y 是 one-hot 编码的真实标签， p 是 Softmax 输出的概率分布。

例题 6.2： 真实标签 $y=1$ （正类），模型预测 $p=0.8$ 。计算二分类交叉熵损失。

解答： $L = -[1 \times \log(0.8) + 0 \times \log(0.2)] = -\log(0.8) = -(-0.2231) = 0.2231$

如果模型预测 $p=0.2$ （严重错误），则 $L = -\log(0.2) = 1.6094$ ，损失大了 7 倍。

6.4 损失函数选择速查

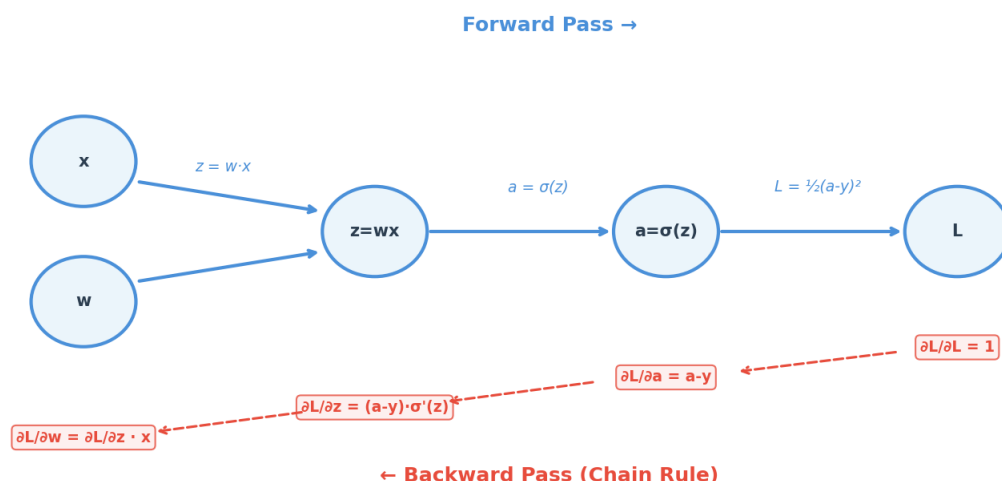
任务类型	损失函数	PyTorch 实现
回归	MSE	nn.MSELoss()
二分类	二分类交叉熵	nn.BCELoss() 或 nn.BCEWithLogitsLoss()
多分类	多分类交叉熵	nn.CrossEntropyLoss()

注意：PyTorch 的 `nn.CrossEntropyLoss()` 内部已经包含了 Softmax，所以网络的最后一层不需要再加 Softmax。

第八章 反向传播：链式法则的工程应用

7.1 反向传播的核心思想

反向传播（Backpropagation）是训练神经网络的核心算法。它的本质就是用链式法则从输出层向输入层逐层计算梯度。



上图展示了一个最简单的神经网络：输入 x 和权重 w 相乘得到 z ，经过 Sigmoid 激活得到 a ，最后计算损失 $L = \frac{1}{2}(a - y)^2$ 。

前向传播（蓝色箭头，从左到右）：

- 1 $z = w \times x$
- 2 $a = \sigma(z)$
- 3 $L = \frac{1}{2}(a - y)^2$

反向传播（红色箭头，从右到左）：

- 4 $\partial L / \partial a = a - y$
- 5 $\partial L / \partial z = \partial L / \partial a \times \sigma'(z) = (a - y) \times \sigma(z)(1 - \sigma(z))$
- 6 $\partial L / \partial w = \partial L / \partial z \times x$

每一步都是链式法则的应用：把"上游梯度"乘以"本地梯度"，传递给"下游"。

7.2 手算反向传播

例题 7.1： 给定 $x=2$, $w=0.5$, $y=1$, 学习率 $\alpha=0.1$ 。执行一次前向传播和反向传播，更新 w 。

前向传播：

$$z = w \times x = 0.5 \times 2 = 1.0$$

$$a = \sigma(1.0) = 1/(1+e^{(-1)}) \approx 0.731$$

$$L = \frac{1}{2}(0.731 - 1)^2 = \frac{1}{2} \times 0.0724 \approx 0.0362$$

反向传播：

$$\partial L / \partial a = 0.731 - 1 = -0.269$$

$$\partial a / \partial z = \sigma(1.0) \times (1 - \sigma(1.0)) = 0.731 \times 0.269 \approx 0.197$$

$$\partial z / \partial w = x = 2$$

$$\partial L / \partial w = \partial L / \partial a \times \partial a / \partial z \times \partial z / \partial w = (-0.269) \times 0.197 \times 2 \approx -0.106$$

更新参数：

$$w_{\text{new}} = 0.5 - 0.1 \times (-0.106) = 0.5 + 0.0106 = \mathbf{0.5106}$$

w 增大了，这是合理的——因为当前预测值 0.731 小于真实值 1，增大 w 会增大 z ，进而增大 a ，使预测更接近真实值。

7.3 PyTorch 中的自动反向传播

在 PyTorch 中，你不需要手动计算梯度——`loss.backward()` 会自动完成反向传播：

```
import torch

import torch.nn as nn

# 前向传播
x = torch.tensor([2.0])
w = torch.tensor([0.5], requires_grad=True)
```

```

y = torch.tensor([1.0])

z = w * x
a = torch.sigmoid(z)
loss = 0.5 * (a - y) ** 2

# 反向传播（自动计算梯度）
loss.backward()

print(f"w.grad = {w.grad.item():.4f}") # 约 -0.106，与手算一致

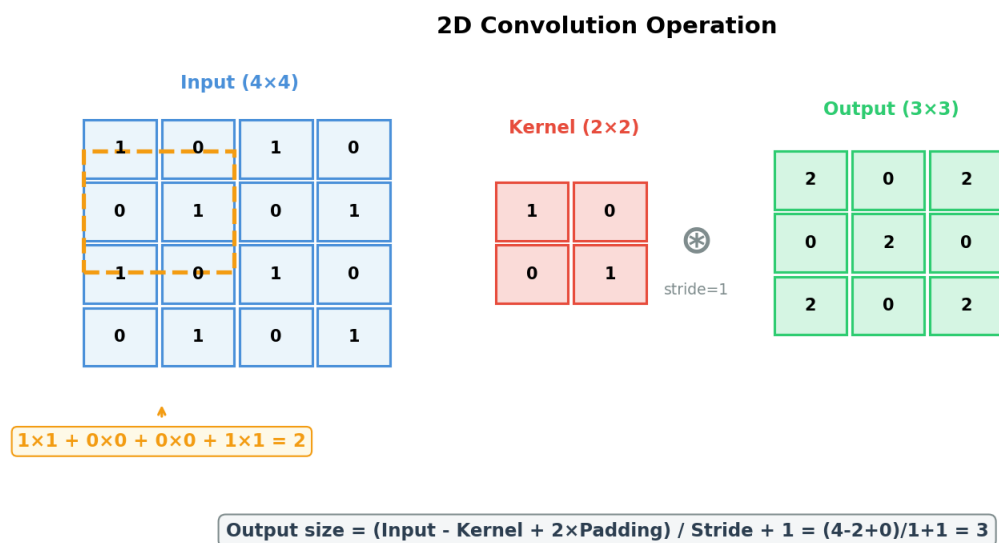
```

PyTorch 的 `autograd` 引擎在前向传播时记录了所有运算的计算图，然后在 `backward()` 时沿着计算图反向应用链式法则。这就是为什么我们需要 `optimizer.zero_grad()`——如果不清零，梯度会累加。

第九章 卷积运算的数学本质

8.1 什么是卷积？

在深度学习中，卷积是一种用小矩阵（卷积核）在大矩阵（输入）上滑动，逐位置计算点积的运算。它是卷积神经网络（CNN）的核心操作。



上图展示了一个 4×4 输入与 2×2 卷积核的卷积过程 (stride=1, padding=0) 。黄色虚线框标出了第一个计算位置：

$$1 \times 1 + 0 \times 0 + 0 \times 0 + 1 \times 1 = 2$$

卷积核在输入上从左到右、从上到下滑动，每个位置计算一次点积，最终得到 3×3 的输出。

8.2 输出尺寸公式

这是 NOAI 考试中的**必考公式**：

$$Output = (Input - Kernel + 2 \times Padding) / Stride + 1$$

参数	含义	常见值
Input	输入特征图的边长	28, 32, 224 等
Kernel	卷积核的边长	3, 5, 7
Padding	边缘填充的像素数	0, 1, 2
Stride	卷积核每次移动的步长	1, 2

例题 8.1：输入图像 32×32，使用 5×5 卷积核，padding=0，stride=1。输出尺寸是多少？

解答： $Output = (32 - 5 + 0) / 1 + 1 = 28$

例题 8.2：输入 28×28，经过 Conv2d(1, 6, 5) → MaxPool2d(2) → Conv2d(6, 16, 5) → MaxPool2d(2)。最终特征图尺寸是多少？（这就是 LeNet-5 的结构）

层	计算	输出尺寸
输入	—	28×28

层	计算	输出尺寸
Conv2d(1,6,5)	$(28-5)/1+1 = 24$	24×24
MaxPool2d(2)	$24/2 = 12$	12×12
Conv2d(6,16,5)	$(12-5)/1+1 = 8$	8×8
MaxPool2d(2)	$8/2 = 4$	4×4

最终特征图尺寸为 **4×4**，通道数为 16，所以展平后的维度为 $16 \times 4 \times 4 = \mathbf{256}$ 。这就是为什么 LeNet-5 的第一个全连接层是 `nn.Linear(16*4*4, 120)`。

8.3 卷积的直觉理解

不同的卷积核能提取不同的特征：

卷积核	功能	效果
<code>[[0,-1,0],[-1,4,-1],[0,-1,0]]</code>	边缘检测	突出图像中的边缘
<code>[[1,1,1],[1,1,1],[1,1,1]] / 9</code>	均值模糊	使图像变模糊
<code>[[0,-1,0],[-1,5,-1],[0,-1,0]]</code>	锐化	增强图像细节

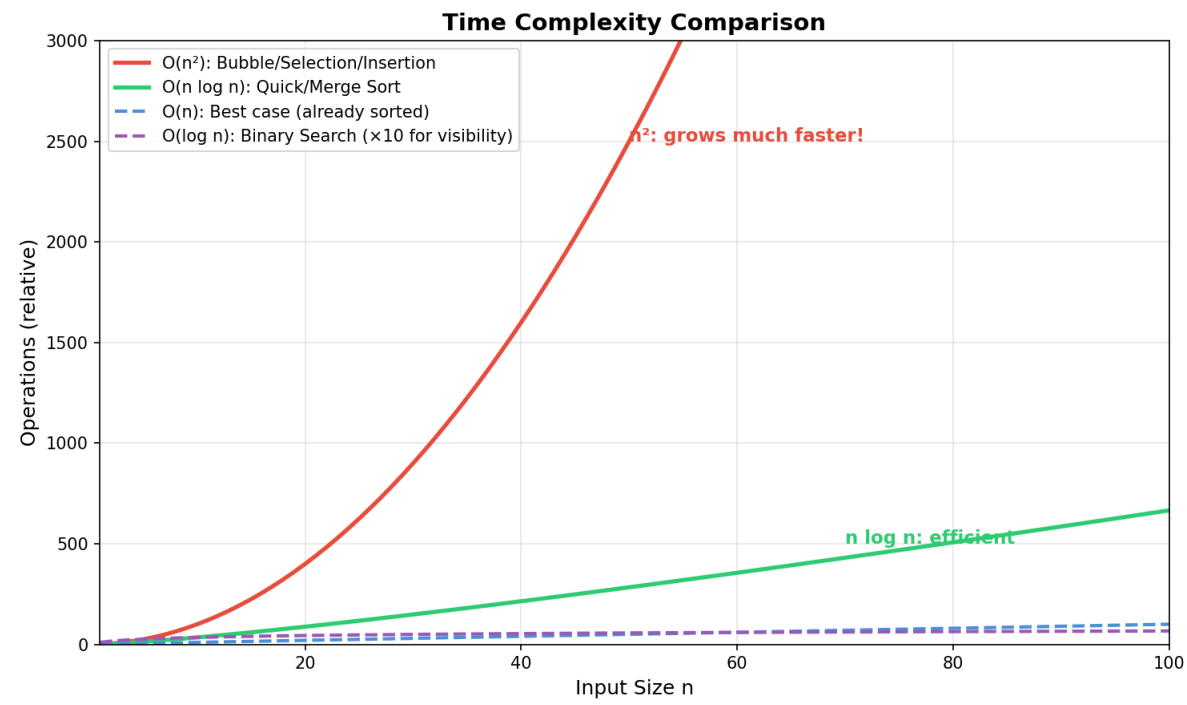
在训练过程中，CNN 会自动学习最适合任务的卷积核参数——浅层学习边缘和纹理，深层学习更抽象的语义特征。

第十章 排序算法与查找算法

排序和查找是计算机科学中最基础的算法问题。在 NOAI 竞赛中，2024 年真题就考察了二分查找的程序补全，而冒泡排序、插入排序等也是高频考点。本章将完整总结四种排序算法和两种查找算法，配合可视化图表和完整的 Python 实现。

10.1 时间复杂度入门

在比较算法之前，我们需要一个衡量算法效率的工具——**时间复杂度**（用大 O 记号表示）。它描述的是随着输入规模 n 的增长，算法执行时间的增长趋势。

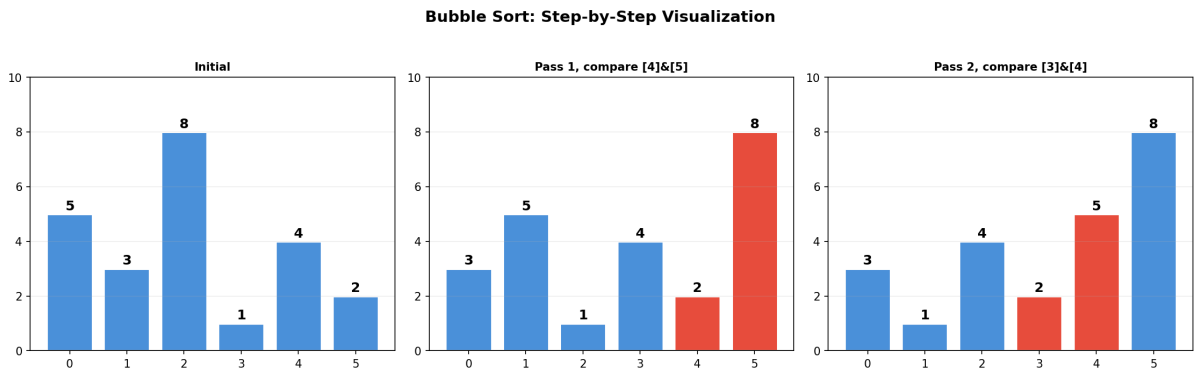


上图对比了几种常见的时间复杂度。可以看到， $O(n^2)$ 的增长速度远远超过 $O(n \log n)$ ，这就是为什么快速排序在实践中比冒泡排序快得多的原因。

复杂度	含义	典型算法	n=1000 时的操作数
$O(1)$	常数时间	数组索引访问	1
$O(\log n)$	对数时间	二分查找	~10
$O(n)$	线性时间	线性查找	1,000
$O(n \log n)$	线性对数时间	快速排序、归并排序	~10,000
$O(n^2)$	平方时间	冒泡排序、选择排序	1,000,000

10.2 冒泡排序 (Bubble Sort)

冒泡排序是最直观的排序算法：反复比较相邻的两个元素，如果顺序错误就交换它们。每一轮遍历都会把当前未排序部分的最大值“冒泡”到末尾。



Python 实现：

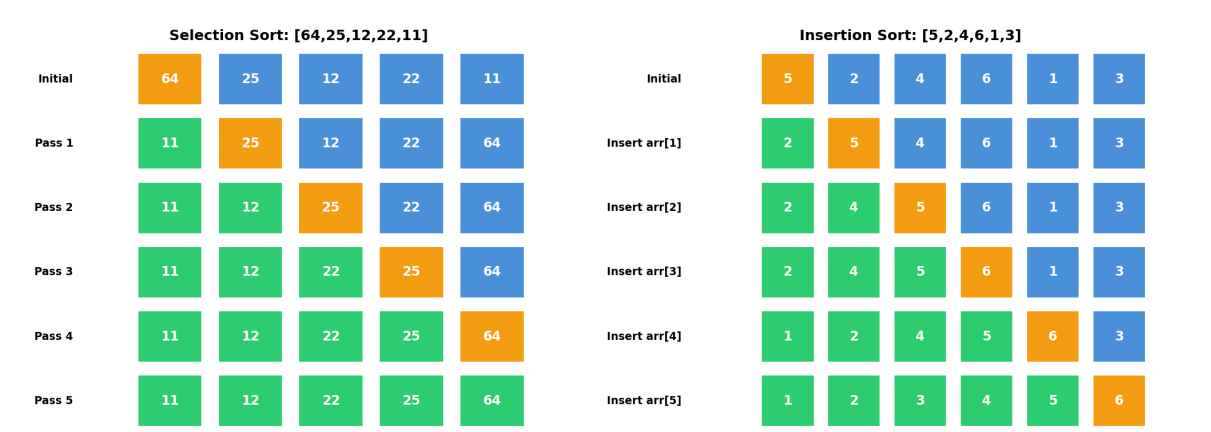
```
def bubble_sort(arr):  
  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n - i - 1):      # 每轮后面 i 个已排好  
            if arr[j] > arr[j + 1]:        # 相邻元素比较  
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # 交换  
                swapped = True  
        if not swapped:                    # 优化：本轮无交换，已有序  
            break  
    return arr  
  
# 示例  
  
print(bubble_sort([5, 3, 8, 1, 4, 2])) # [1, 2, 3, 4, 5, 8]
```

执行过程跟踪（以 [5, 3, 8, 1] 为例）：

轮次	比较过程	结果
第 1 轮	5>3→换, 5<8→不换, 8>1→换	[3, 5, 1, 8]
第 2 轮	3<5→不换, 5>1→换	[3, 1, 5, 8]
第 3 轮	3>1→换	[1, 3, 5, 8]
指标	复杂度	
最好情况	O(n) (已有序, 加了 swapped 优化)	
平均情况	O(n²)	
最坏情况	O(n²) (逆序)	
空间复杂度	O(1) (原地排序)	
稳定性	稳定 (相同元素不会交换顺序)	

10.3 选择排序 (Selection Sort)

选择排序的思路是：每次从未排序部分找到最小的元素，放到已排序部分的末尾。



上图左侧展示了选择排序的每一轮状态：绿色表示已排序部分，橙色表示当前正在处理的位置。

Python 实现：

```
def selection_sort(arr):  
  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i + 1, n):           # 在未排序部分找最小值  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  # 放到已排序末尾  
    return arr  
  
print(selection_sort([64, 25, 12, 22, 11]))  # [11, 12, 22, 25, 64]
```

执行过程跟踪（以 [64, 25, 12, 22, 11] 为例）：

轮次	找到的最小值	交换位置	结果
第 1 轮	11（索引 4）	[0]↔4	[11, 25, 12, 22, 64]
第 2 轮	12（索引 2）	[1]↔2	[11, 12, 25, 22, 64]
第 3 轮	22（索引 3）	[2]↔3	[11, 12, 22, 25, 64]
第 4 轮	25（索引 3）	无需交换	[11, 12, 22, 25, 64]
指标	复杂度		
所有情况	O(n²)（始终需要扫描未排序部分）		
空间复杂度	O(1)		
稳定性	不稳定		

10.4 插入排序（Insertion Sort）

插入排序的思路类似打牌：每次拿起一张新牌，插入到已排好的牌中的正确位置。

上图右侧展示了插入排序的过程：每一步取出一个元素（橙色），向左找到它应该插入的位置。

Python 实现：

```
def insertion_sort(arr):  
  
    for i in range(1, len(arr)):  
        key = arr[i]                # 拿起“新牌”  
        j = i - 1  
        while j >= 0 and arr[j] > key:  # 向左找插入位置  
            arr[j + 1] = arr[j]        # 元素右移  
            j -= 1  
        arr[j + 1] = key              # 插入  
    return arr  
  
print(insertion_sort([5, 2, 4, 6, 1, 3])) # [1, 2, 3, 4, 5, 6]
```

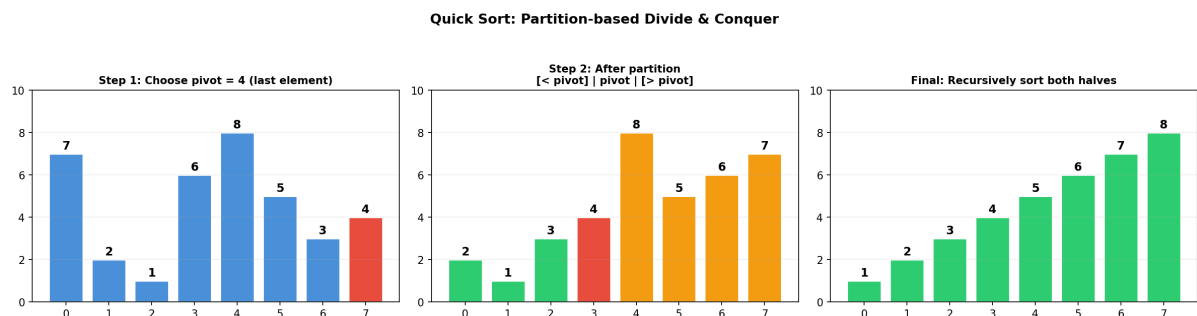
执行过程跟踪（以 [5, 2, 4, 6, 1, 3] 为例）：

步骤	拿起的牌	插入位置	结果
i=1	2	插入到 5 前面	[2, 5, 4, 6, 1, 3]
i=2	4	插入到 2 和 5 之间	[2, 4, 5, 6, 1, 3]
i=3	6	已在正确位置	[2, 4, 5, 6, 1, 3]
i=4	1	插入到最前面	[1, 2, 4, 5, 6, 3]
i=5	3	插入到 2 和 4 之间	[1, 2, 3, 4, 5, 6]
指标	复杂度		
最好情况	O(n)（已有序）		

步骤	拿起的牌	插入位置	结果
平均/最坏	$O(n^2)$		
空间复杂度	$O(1)$		
稳定性	稳定		

10.5 快速排序 (Quick Sort)

快速排序是实践中最常用的排序算法，采用分治策略：选择一个“基准元素” (pivot)，把数组分成“小于 pivot”和“大于 pivot”两部分，然后递归地排序两部分。



Python 实现：

```
def quick_sort(arr):

    if len(arr) <= 1:
        return arr
    pivot = arr[-1] # 选择最后一个元素作为基准
    left = [x for x in arr[:-1] if x <= pivot] # 小于等于 pivot
    right = [x for x in arr[:-1] if x > pivot] # 大于 pivot
    return quick_sort(left) + [pivot] + quick_sort(right)

print(quick_sort([7, 2, 1, 6, 8, 5, 3, 4])) # [1, 2, 3, 4, 5, 6, 7, 8]
```

原地版本（更接近考试中的写法）：

```
def quick_sort_inplace(arr, low, high):
```

```
if low < high:
    pivot_idx = partition(arr, low, high)
    quick_sort_inplace(arr, low, pivot_idx - 1)
    quick_sort_inplace(arr, pivot_idx + 1, high)

def partition(arr, low, high):
    pivot = arr[high] # 基准元素
    i = low - 1 # 小于 pivot 的区域的右边界
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

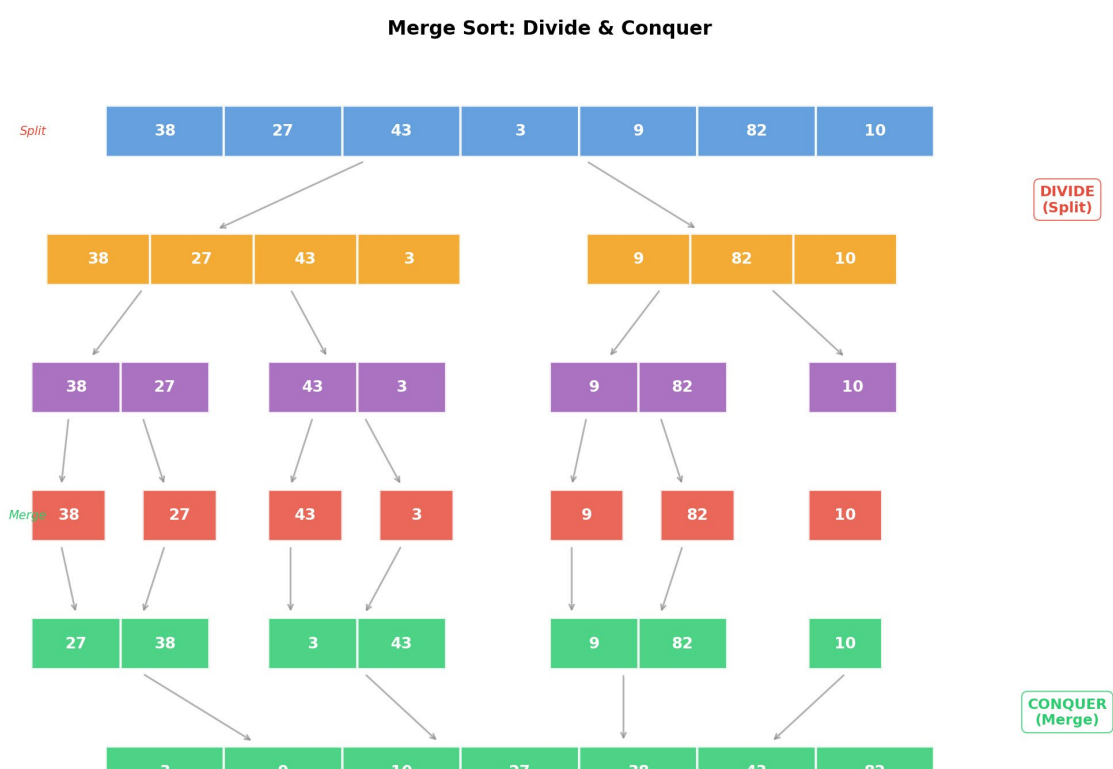
data = [7, 2, 1, 6, 8, 5, 3, 4]
quick_sort_inplace(data, 0, len(data) - 1)

print(data) # [1, 2, 3, 4, 5, 6, 7, 8]
```

指标	复杂度
最好/平均	$O(n \log n)$
最坏情况	$O(n^2)$ （每次选到最大/最小值作为 pivot）
空间复杂度	$O(\log n)$ （递归栈）
稳定性	不稳定

10.6 归并排序（Merge Sort）

归并排序也采用分治策略，但与快速排序不同的是，它的“分”非常简单（从中间一切两半），而“治”的过程（合并两个有序数组）才是核心。归并排序的最大优势是：无论输入什么数据，时间复杂度始终是 $O(n \log n)$ ，不会像快速排序那样退化到 $O(n^2)$ 。



上图展示了归并排序的完整过程：先将数组 [38, 27, 43, 3, 9, 82, 10] 递归地拆分到单个元素（红色），然后逐层合并成有序数组（绿色）。

合并操作的核心思路：给定两个已排序的数组，用两个指针分别指向它们的开头，每次取较小的那个放入结果数组，然后移动对应指针。

合并过程示例（合并 [3, 27, 38, 43] 和 [9, 10, 82]）：

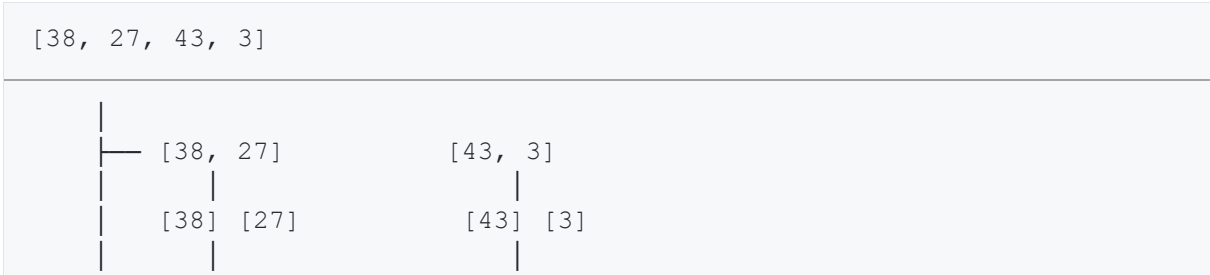
步骤	左指针	右指针	比较	结果数组
1	3	9	$3 < 9 \rightarrow$ 取 3	[3]
2	27	9	$27 > 9 \rightarrow$ 取 9	[3, 9]
3	27	10	$27 > 10 \rightarrow$ 取 10	[3, 9, 10]
4	27	82	$27 < 82 \rightarrow$ 取 27	[3, 9, 10, 27]
5	38	82	$38 < 82 \rightarrow$ 取 38	[3, 9, 10, 27, 38]

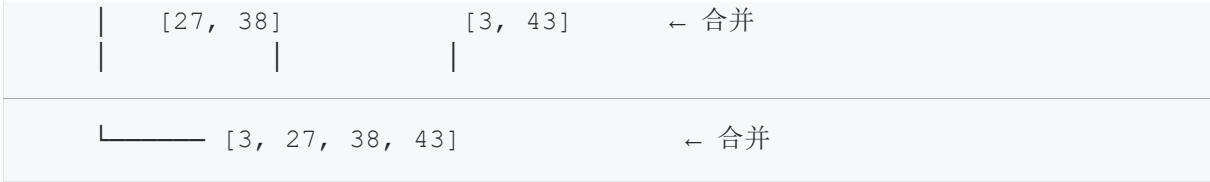
步骤	左指针	右指针	比较	结果数组
6	43	82	43 < 82 → 取 43	[3, 9, 10, 27, 38, 43]
7	已用完	82	取剩余	[3, 9, 10, 27, 38, 43, 82]

Python 实现：

```
def merge_sort(arr):  
  
    if len(arr) <= 1:  
        return arr  
    mid = len(arr) // 2  
    left = merge_sort(arr[:mid])           # 递归排序左半  
    right = merge_sort(arr[mid:])          # 递归排序右半  
    return merge(left, right)              # 合并两个有序数组  
  
def merge(left, right):  
    result = []  
    i = j = 0  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:             # 注意是 <=, 保证稳定性  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    result.extend(left[i:])                  # 拼接剩余元素  
    result.extend(right[j:])  
    return result  
  
print(merge_sort([38, 27, 43, 3, 9, 82, 10])) # [3, 9, 10, 27, 38, 43, 82]
```

执行过程跟踪（以 [38, 27, 43, 3] 为例）：





指标	复杂度
所有情况	$O(n \log n)$ （始终稳定，不会退化）
空间复杂度	$O(n)$ （需要额外数组存储合并结果）
稳定性	稳定（相同元素保持原始顺序）

归并排序 vs 快速排序：

特性	归并排序	快速排序
最坏时间	$O(n \log n)$ ✓	$O(n^2)$ ⚠
空间	$O(n)$ ⚠	$O(\log n)$ ✓
稳定性	稳定 ✓	不稳定 ⚠
实践速度	稍慢（常数因子大）	更快（缓存友好）

10.7 五种排序算法对比总结

算法	最好	平均	最坏	空间	稳定	特点
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	最简单，适合教学
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	交换次数最少
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	小数据/近似有序时最快

算法	最好	平均	最坏	空间	稳定	特点
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定	实践中最常用
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定	最坏情况也不退化

*考试提示: Python 内置的 `sorted()` 和 `list.sort()` 使用的是 **Timsort** 算法 (结合了插入排序和归并排序的优点), 时间复杂度为 $O(n \log n)$, 且是稳定的。*

10.8 线性查找 (Linear Search)

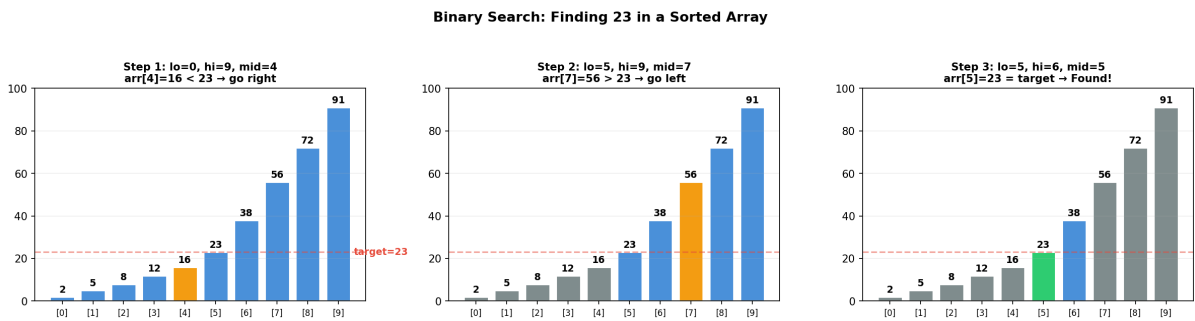
线性查找是最简单的查找方法: 从头到尾逐个检查每个元素, 直到找到目标或遍历完整个数组。

```
def linear_search(arr, target):  
  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i          # 返回索引  
    return -1                 # 未找到  
  
arr = [4, 2, 7, 1, 9, 3]  
print(linear_search(arr, 7))  # 2 (找到, 索引为 2)  
  
print(linear_search(arr, 5))  # -1 (未找到)
```

指标	复杂度
最好情况	$O(1)$ (第一个就是)
平均/最坏	$O(n)$
要求	无 (数组不需要有序)

10.9 二分查找 (Binary Search)

二分查找是一种高效的查找算法，但要求数组必须已排序。它的思路是：每次取中间元素与目标比较，将搜索范围缩小一半。



上图展示了在有序数组 [2, 5, 8, 12, 16, 23, 38, 56, 72, 91] 中查找 23 的过程：只需 3 步就找到了目标，而线性查找需要 6 步。

Python 实现（这是 2024 年真题的考察内容）：

```
def binary_search(arr, target):  
  
    low = 0  
    high = len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1  
  
arr = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]  
print(binary_search(arr, 23)) # 5  
  
print(binary_search(arr, 10)) # -1
```

执行过程跟踪（查找 23）：

步骤	low	high	mid	arr[mid]	比较结果
1	0	9	4	16	$16 < 23 \rightarrow \text{low} = 5$
2	5	9	7	56	$56 > 23 \rightarrow \text{high} = 6$
3	5	6	5	23	$23 = 23 \rightarrow$ 找到!

关键细节（考试易错点）：

易错点	正确写法	错误写法	后果
循环条件	<u>$\text{while low} \leq \text{high}$</u>	<u>$\text{while low} < \text{high}$</u>	漏查单元素
中间索引	<u>$\text{mid} = (\text{low} + \text{high}) // 2$</u>	<u>$\text{mid} = (\text{low} + \text{high}) / 2$</u>	浮点数索引错误
边界更新	<u>$\text{low} = \text{mid} + 1$</u>	<u>$\text{low} = \text{mid}$</u>	可能无限循环
边界更新	<u>$\text{high} = \text{mid} - 1$</u>	<u>$\text{high} = \text{mid}$</u>	可能无限循环

指标	复杂度
所有情况	$O(\log n)$
空间复杂度	$O(1)$ （迭代版）
要求	数组必须已排序

为什么二分查找这么快？ 每次比较都将搜索范围缩小一半。对于 $n=1,000,000$ 的数组，线性查找最多需要 1,000,000 次比较，而二分查找只需要 $\log_2(1,000,000) \approx 20$ 次。

10.10 查找算法对比

算法	时间复杂度	要求有序	适用场景
线性查找	$O(n)$	否	小数据、无序数据
二分查找	$O(\log n)$	是	大数据、已排序数据

例题 10.1：对数组 [1, 3, 5, 7, 9, 11, 13, 15] 用二分查找查找 7，写出每步的 low、high、mid 值。

解答：

步骤	low	high	mid	arr[mid]	结果
1	0	7	3	7	$7 = 7 \rightarrow$ 找到，返回 3

运气好的情况！第一次就命中了。

例题 10.2：对数组 [2, 4, 6, 8, 10, 12] 用二分查找查找 5（不存在），写出每步的过程。

解答：

步骤	low	high	mid	arr[mid]	结果
1	0	5	2	6	$6 > 5 \rightarrow \text{high} = 1$
2	0	1	0	2	$2 < 5 \rightarrow \text{low} = 1$
3	1	1	1	4	$4 < 5 \rightarrow \text{low} = 2$
4	2	1	—	—	$\text{low} > \text{high} \rightarrow$ 未找到，返回 -1

注意第 4 步： $\text{low}=2 > \text{high}=1$ ，循环结束，说明目标不存在。

综合练习题

练习 1：概率计算

一个二分类模型的混淆矩阵如下：

	预测正	预测负
实际正	80 (TP)	20 (FN)
实际负	10 (FP)	90 (TN)

请计算：(a) 精确率 (b) 召回率 (c) F1 分数 (d) 准确率

解答：

(a) $\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 80 / (80 + 10) = \mathbf{0.889}$

(b) $\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 80 / (80 + 20) = \mathbf{0.800}$

(c) $\text{F1} = 2 \times \text{P} \times \text{R} / (\text{P} + \text{R}) = 2 \times 0.889 \times 0.800 / (0.889 + 0.800) = \mathbf{0.842}$

(d) $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{TN} + \text{FN}) = (80 + 90) / 200 = \mathbf{0.850}$

练习 2：梯度下降手算

损失函数 $L(w) = 2w^2 - 4w + 3$ ，初始 $w_0 = 3$ ，学习率 $\alpha = 0.2$ 。

(a) 求 $L'(w)$

(b) 手算前 3 步梯度下降

(c) 理论最小值点在哪里？

解答：

(a) $L'(w) = 4w - 4$

(b)

步骤	w	L(w)	L'(w)	w_new
0	3.0	9.0	8.0	$3.0 - 0.2 \times 8.0 = \mathbf{1.4}$
1	1.4	1.32	1.6	$1.4 - 0.2 \times 1.6 = \mathbf{1.08}$
2	1.08	1.0128	0.32	$1.08 - 0.2 \times 0.32 = \mathbf{1.016}$

(c) 令 $L'(w) = 4w - 4 = 0$ ，得 $w = \mathbf{1.0}$ ， $L(1) = 2 - 4 + 3 = \mathbf{1.0}$ 。可以看到梯度下降正在逐步逼近这个最小值。

练习 3：链式法则与反向传播

给定简单网络： $z = 2x + 1$ ， $a = \text{ReLU}(z)$ ， $L = (a - 3)^2$ 。当 $x = 2$ 时：

(a) 计算前向传播的所有中间值

(b) 计算 $\partial L / \partial x$

解答：

(a) $z = 2 \times 2 + 1 = 5$ ， $a = \text{ReLU}(5) = 5$ ， $L = (5 - 3)^2 = 4$

(b) 链式法则：

$$\partial L / \partial a = 2(a - 3) = 2(5 - 3) = 4$$

$$\partial a / \partial z = 1 \quad (\text{因为 } z = 5 > 0, \text{ReLU 导数为 } 1)$$

$$\partial z / \partial x = 2$$

$$\partial L / \partial x = \partial L / \partial a \times \partial a / \partial z \times \partial z / \partial x = 4 \times 1 \times 2 = \mathbf{8}$$

练习 4：卷积尺寸推导

一个 CNN 的结构如下：

输入: $32 \times 32 \times 3$
$\text{Conv2d}(3, 32, 3, \text{padding}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2)$ $\text{Conv2d}(32, 64, 3, \text{padding}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2)$
$\text{Flatten} \rightarrow \text{Linear}(?, 10)$

(a) 计算每一层的输出尺寸

(b) Linear 层的输入维度 ? 应该是多少?

解答:

层	计算	输出尺寸
输入	—	$32 \times 32 \times 3$
$\text{Conv2d}(3, 32, 3, p=1)$	$(32-3+2)/1+1=32$	$32 \times 32 \times 32$
$\text{MaxPool2d}(2)$	$32/2=16$	$16 \times 16 \times 32$
$\text{Conv2d}(32, 64, 3, p=1)$	$(16-3+2)/1+1=16$	$16 \times 16 \times 64$
$\text{MaxPool2d}(2)$	$16/2=8$	$8 \times 8 \times 64$
Flatten	$8 \times 8 \times 64$	4096

(b) Linear 层的输入维度为 **4096**, 即 `nn.Linear(4096, 10)`。

练习 5: Softmax 与交叉熵

模型输出 $\text{logits} = [3.0, 1.0, 0.5]$, 真实标签为第 0 类 (即 $y = [1, 0, 0]$)。

(a) 计算 Softmax 概率

(b) 计算交叉熵损失

解答:

(a) $e^{3.0} = 20.086$, $e^{1.0} = 2.718$, $e^{0.5} = 1.649$, 总和 = 24.453

Softmax = $[20.086/24.453, 2.718/24.453, 1.649/24.453] = [0.821, 0.111, 0.067]$

(b) $L = -[1 \times \log(0.821) + 0 \times \log(0.111) + 0 \times \log(0.067)] = -\log(0.821) = 0.197$

附录 公式速查表

A. 概率与统计

公式	表达式	适用场景
条件概率	$P(A B) = P(A \cap B) / P(B)$	已知 B 发生，求 A 的概率
贝叶斯定理	$P(A B) = P(B A)P(A) / P(B)$	从结果推原因
均值	$\mu = (1/n)\sum x_i$	数据中心
方差	$\sigma^2 = (1/n)\sum (x_i - \mu)^2$	数据离散程度
标准化	$z = (x - \mu) / \sigma$	数据预处理
CDF	$F(x) = P(X \leq x)$	累积概率
$P(a < X \leq b)$	$F(b) - F(a)$	区间概率
二项分布	$P(X=k) = C(n,k)p^k(1-p)^{n-k}$	成功次数概率
二项均值/方差	$\mu=np, \sigma^2=np(1-p)$	二项分布参数
均匀分布 PDF	$f(x) = 1/(b-a)$	等概率密度
均匀分布均值	$\mu = (a+b)/2$	区间中点

B. 求导公式

函数	导数	备注
x^n	$nx^{(n-1)}$	幂函数
e^x	e^x	自然指数
$\ln(x)$	$1/x$	自然对数
$\sigma(x)$	$\sigma(x)(1-\sigma(x))$	Sigmoid
$\text{ReLU}(x)$	$0\ (x<0),\ 1\ (x>0)$	分段
链式法则	$dy/dx = dy/du \times du/dx$	复合函数

C. 梯度下降

公式	表达式	说明
参数更新	$\theta = \theta - \alpha \nabla L$	α 为学习率
MSE 梯度	$(2/n)\Sigma(\hat{y}-y)x$	线性回归
交叉熵梯度	$(1/n)\Sigma(\hat{y}-y)x$	逻辑回归

D. 卷积尺寸

公式	表达式
卷积输出	$(\text{Input} - \text{Kernel} + 2\text{Padding}) / \text{Stride} + 1$
池化输出	$\text{Input} / \text{PoolSize}$

公式	表达式
展平维度	$\text{Channels} \times \text{Height} \times \text{Width}$

E. 排序算法

算法	最好	平均	最坏	空间	稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	否
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	是

F. 查找算法

算法	时间复杂度	要求有序	关键细节
线性查找	$O(n)$	否	逐个遍历
二分查找	$O(\log n)$	是	while low <= high, mid = (low+high)//2

本教程由 Johnny Cheng 团队编写，配合 NOAI 2026 其他备赛资源使用。数学不是障碍，而是理解 AI 的钥匙。祝你备考顺利！